



## Fundamentos de Sistemas Embebidos

Facultad de Ingeniería

UNAM

# Laboratorio Salas A y B

---

*Profesor(a):* Eduardo Fragoso Navarro

*Asignatura:* Fundamentos de Sistemas Embebidos

*Grupo:* 3

*Número de Práctica(s):* 1

*Integrante(s):* Tepal Briseño Hansel Yael - 320078282

Ugartechea González Luis Antonio - 320223822

*Número de brigada:* 10

*Semestre:* 9°

*Fecha de entrega:* 9 de septiembre del 2026

*Observaciones:* \_\_\_\_\_  
\_\_\_\_\_

CALIFICACIÓN: \_\_\_\_\_

# Conversión Analógica-Digital, PWM e Interfaz Gráfica

## Objetivo

Desarrollar un sistema embebido utilizando la tarjeta ESP32-S3 que integre la lectura de un potenciómetro a través del convertidor analógico-digital (ADC) para controlar el brillo de un LED mediante una señal PWM y un transistor MOSFET, visualizando simultáneamente los datos obtenidos en valor absoluto, porcentaje y representación gráfica en una pantalla OLED vía I2C; aplicando además buenas prácticas de programación al estructurar el código fuente en archivos separados (encabezados e implementación) para mejorar la modularidad del proyecto.

## Resultados

### Ejercicio 1

Se llevó a cabo la implementación física del circuito esquemático sobre la protoboard. En la siguiente imagen se presenta el armado final, donde se ve la integración de la tarjeta de desarrollo ESP32-S3 con el potenciómetro, el led y la pantalla OLED. Se puede observar la conexión del potenciómetro empleado para la lectura de la señal analógica, el cableado de la pantalla OLED hacia las terminales del bus I2C, y la configuración de la etapa de potencia conformada por el transistor MOSFET 2N7000, el cual se encarga de accionar el LED a partir de la señal PWM.

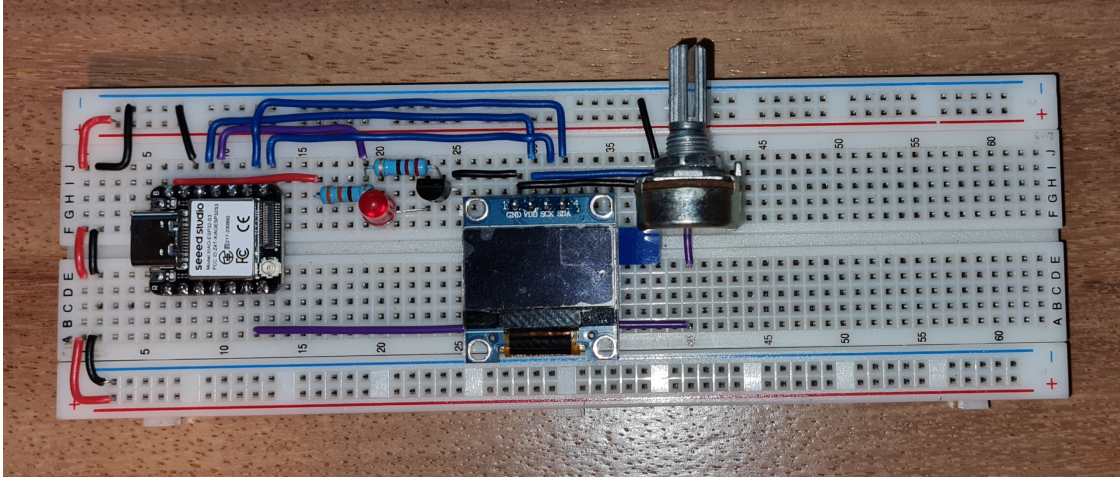


Figure 1: Circuito final implementado en la protoboard.

### Ejercicio 2

Para el segundo ejercicio, se desarrolló el código encargado de adquirir los datos del convertidor analógico-digital (ADC) y generar la señal PWM correspondiente para el control de iluminación del LED. Adicionalmente, se implementó la interfaz gráfica en la pantalla OLED utilizando la

librería u8g2. En la siguiente imagen se muestra el funcionamiento del programa, donde se aprecia el despliegue de los tres elementos requeridos, el valor directo del ADC en un rango de 0 a 4095, su equivalencia porcentual (de 0 a 100%) y la representación gráfica dinámica que ilustra la lectura actual del potenciómetro. Lo más relevante dentro del código:

```
1  void app_main(void)
2  {
3      pantalla_iniciar();
4      adc_iniciar();
5      pwm_iniciar();
6
7      while (1)
8      {
9          int lectura = adc_leer();
10
11          pwm_actualizar(lectura);
12          pantalla_mostrar(lectura);
13
14          vTaskDelay(pdMS_TO_TICKS(PERIODO_MS));
15      }
16 }
```

Listing 1: Ciclo principal del programa.

```
1  int porcentaje = (lectura * 100) / ADC_MAX;
2  // ... (Configuración de texto)
3  snprintf(texto, sizeof(texto), "ADC: %04d | %3d%%", lectura,
4           porcentaje);
5
6  // Dibujo de la barra de progreso
7  u8g2_DrawFrame(&u8g2, BARRA_X, BARRA_Y, BARRA_ANCHO, BARRA_ALTO)
8  ;
9  if (porcentaje > 0) {
10     u8g2_DrawBox(&u8g2, BARRA_X, BARRA_Y, porcentaje, BARRA_ALTO)
11     ;
12 }
13 }
```

Listing 2: Función para mostrar los datos en la pantalla OLED.

Con este código obtenemos los siguientes resultados en la pantalla OLED, donde se puede observar el valor del ADC, su porcentaje equivalente y la barra de progreso que representa gráficamente la lectura actual del potenciómetro.

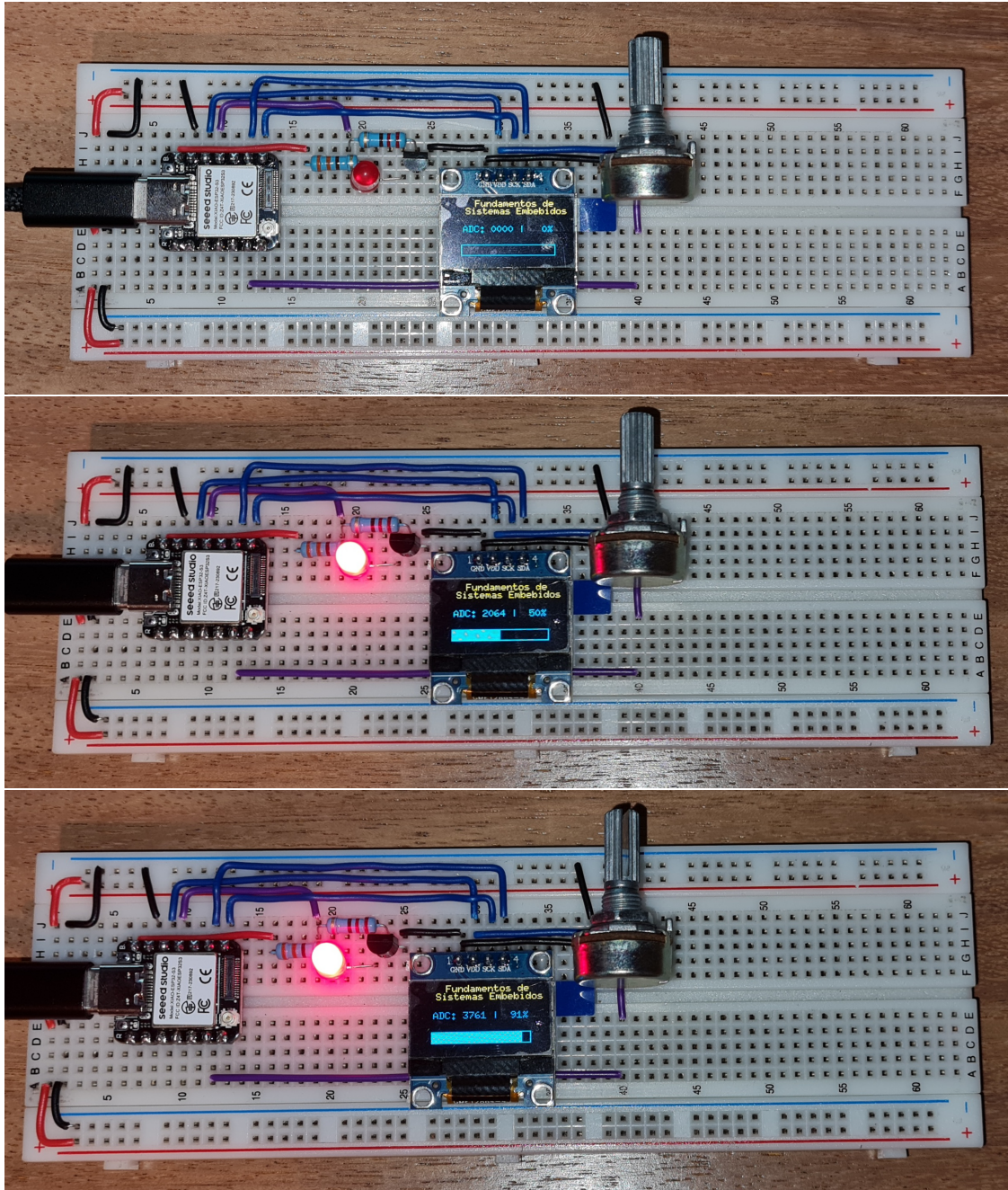


Figure 2: Circuito funcionando en la protoboard.

### Ejercicio 3

Dado que el comportamiento visual y funcional del sistema se mantiene exactamente igual que en el Ejercicio 2, el enfoque de este apartado se centra exclusivamente en la reestructuración del código. A diferencia de la implementación anterior, el proyecto se dividió en tres archivos clave

para mejorar su modularidad: un archivo de encabezado (`perifericos.h`) para las declaraciones y macros, un archivo de implementación (`perifericos.c`) con el cuerpo de las funciones, y el archivo principal (`main.c`) enfocado únicamente en la ejecución de la función principal. De lo más relevante dentro del código tenemos `perifericos.h`, que destaca el uso de `#pragma once` para evitar inclusiones múltiples y el uso de `extern` para compartir los handlers sin redefinirlos.

```
1 #pragma once
2
3 #include "u8g2.h"
4 #include "esp_adc/adc_oneshot.h"
5
6 // ... (Definición de constantes y macros)
7
8 // Declaracion de handlers (extern)
9 extern u8g2_t u8g2;
10 extern adc_oneshot_unit_handle_t adc;
11
12 // Prototipos de funciones
13 void pantalla_iniciar(void);
14 void pantalla_mostrar(int lectura);
15 // ...
```

Listing 3: Archivo de encabezado `perifericos.h`.

Y el archivo principal `main.c` que se centra en la función principal del programa, manteniendo la lógica de adquisición de datos y actualización de la pantalla.

```
1 #include "perifericos.h"
2 #include "freertos/FreeRTOS.h"
3 #include "freertos/task.h"
4
5 void app_main(void)
6 {
7     // La lógica de inicialización y el ciclo infinito
8     // se mantienen idénticos, pero el código ahora
9     // reside en perifericos.c
10    // ...
```

Listing 4: Archivo principal `main.c`.

Con esto obtenemos el mismo resultado visual y funcional que en el Ejercicio 2, pero con un código más organizado y modular, facilitando su mantenimiento y escalabilidad.

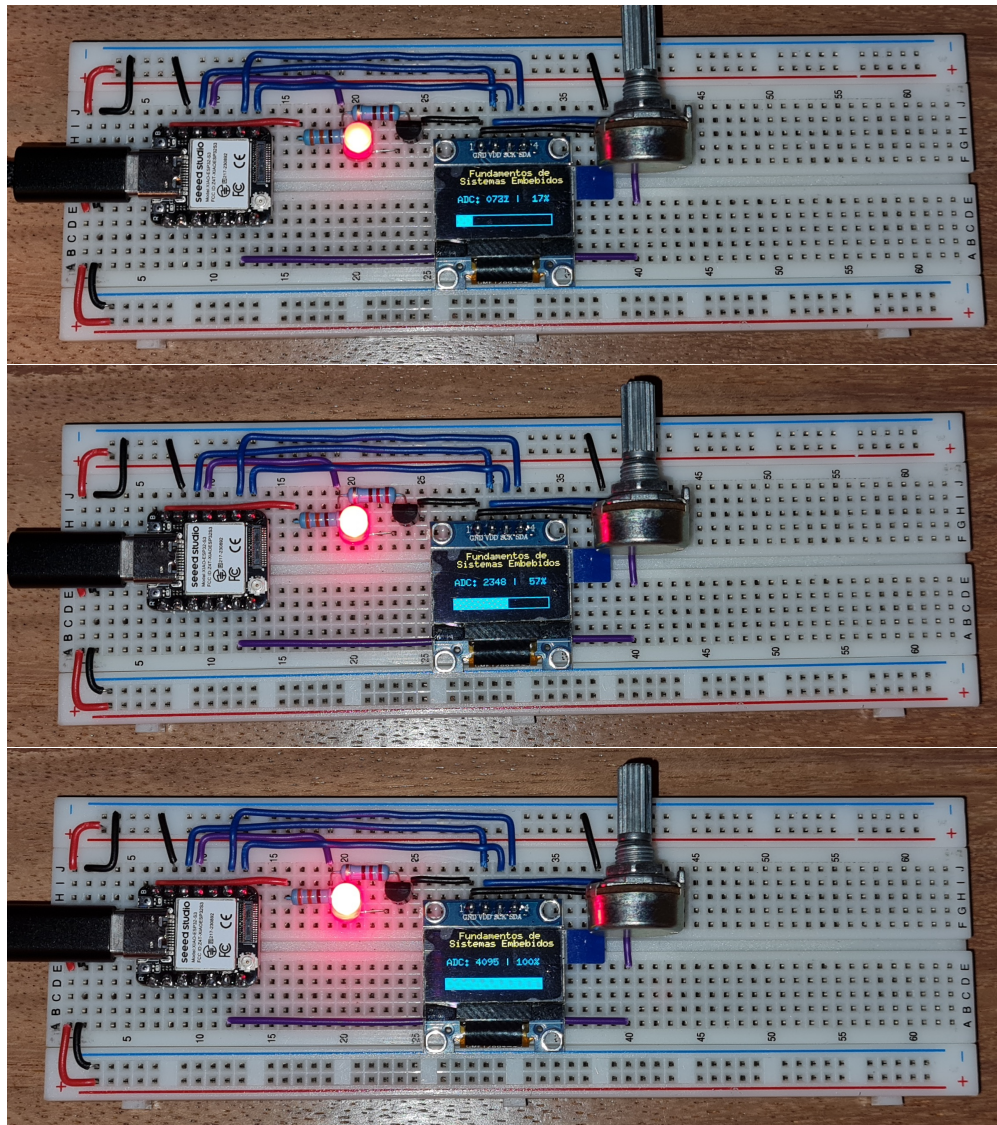


Figure 3: Circuito funcionando en la protoboard.

## Análisis

Las variables `ADC_BITWIDTH_12` y `ADC_ATTEN_DB_12` son fundamentales para establecer una resolución de 12 bits, otorgando un rango de valores de 0 a 4095, y aplicar una atenuación que permite medir voltajes de hasta  $\approx 3.3V$  en la tarjeta de desarrollo. Alterar esto afectaría directamente la medición, por ejemplo, reducir la atenuación a 0 dB saturaría la lectura con voltajes cercanos a 1.1V, mientras que disminuir el ancho de bits restaría precisión y granularidad a los datos obtenidos. Durante las lecturas, es común observar ruido en los valores, un fenómeno que afecta a todos los microcontroladores debido al ruido eléctrico interno y a las interferencias electromagnéticas captadas por el cableado del circuito.

Por otro lado, la temporización y las frecuencias de actualización juegan un papel crítico en el control de actuadores y la visualización gráfica. Modificar considerablemente la frecuencia de la señal PWM generaría problemas perceptibles:

- Si la frecuencia es demasiado baja, el LED podría parpadear visiblemente, afectando la experiencia del usuario.
- Si la frecuencia es excesivamente alta, la capacitancia parásita del MOSFET podría impedir una conmutación limpia, provocando distorsión en la señal o sobrecalentamiento del componente.

En cuanto a la interfaz gráfica, la velocidad de actualización de la pantalla podría optimizarse a nivel de hardware aumentando la frecuencia del reloj del bus I2C; y, a nivel de software, refrescando únicamente los elementos dinámicos, como la barra de progreso, en lugar de limpiar y reescribir todo el buffer del display en cada iteración.

## Conclusiones

El desarrollo demuestra la versatilidad del sistema construido, ya que la arquitectura base puede extrapolarse a diversas aplicaciones prácticas. El potenciómetro, utilizado como entrada analógica, puede ser sustituido por sensores como fotorresistores (LDR) para medir intensidad luminosa o joysticks para controles direccionales. De igual forma, el LED puede reemplazarse por otros actuadores de corriente continua, como motores pequeños, ventiladores o zumbadores, aprovechando el transistor MOSFET para manejar mayores exigencias de corriente. Para garantizar un funcionamiento óptimo en cualquier escenario, es indispensable mitigar el ruido en el ADC implementando soluciones de hardware, como agregar capacitores de desacoplo para filtrar la señal, y soluciones de software, mediante el cálculo de promedios móviles que estabilicen las lecturas.

Finalmente, desde la perspectiva del desarrollo de software, el proyecto resalta la importancia de estructurar el código fuente de manera adecuada. Separar el programa utilizando un archivo de encabezado (`perifericos.h`) para las macros y declaraciones, junto con un archivo de implementación (`perifericos.c`) para la lógica de los controladores, otorga ventajas significativas frente a concentrar todo en el `main.c`. Esta modularización facilita la lectura y el mantenimiento del proyecto al ocultar los detalles de bajo nivel, además de permitir la reutilización de los componentes programados en futuros desarrollos de manera eficiente.

**NOTA:** Si se desea consultar el código fuente completo del proyecto, se puede acceder al repositorio de GitHub en la siguiente dirección: [https://github.com/lethalSopape/r/laboratorio\\_sistemas\\_embebidos](https://github.com/lethalSopape/r/laboratorio_sistemas_embebidos)